

Test vs Mutant: Adversarial LLM Agents for Robust Unit Test Generation

PENGYU CHANG*, Shanghai Jiao Tong University, China

YIXIONG FANG*, Shanghai Jiao Tong University, China

SILIN CHEN, Shanghai Jiao Tong University, China

YULING SHI, Shanghai Jiao Tong University, China

BEIJUN SHEN, Shanghai Jiao Tong University, China

XIAODONG GU[✉], Shanghai Jiao Tong University, China

Software testing is a critical, yet resource-intensive phase of the software development lifecycle. Search-based approaches typically achieve high coverage but produce tests with low readability, whereas large language model (LLM)-based methods generate more human-readable tests but often suffer from low coverage and compilability. While the majority of research efforts have focused on improving test coverage and readability, comparatively less attention has been paid to enhancing the robustness of bug detection. To address this gap, we propose ADVERTEST, a novel adversarial framework for LLM-powered test case generation that pairs a test case generation agent ($\mathcal{T}$) with a mutant generation agent ($\mathcal{M}$): $\mathcal{M}$ persistently creates mutants “hacking” the blind spots of $\mathcal{T}$ ’s current test suite, while $\mathcal{T}$ iteratively refines its tests to “kill” the challenging mutants, with the interaction guided by both coverage and mutation scores. Experimental results on Defects4J show that our approach improves fault detection rates by 8.56% over the best existing LLM-based methods and by 50.20% over EvoSuite, while remaining competitive on line and branch coverage.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → **Artificial intelligence**.

Additional Key Words and Phrases: Automated Test Generation, Large Language Models, Unit Testing, Mutation Testing

ACM Reference Format:

Pengyu Chang, Yixiong Fang, Silin Chen, Yuling Shi, Beijun Shen, and Xiaodong Gu. 2026. Test vs Mutant: Adversarial LLM Agents for Robust Unit Test Generation. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA031 (October 2026), 24 pages. <https://doi.org/10.1145/3832122>

1 Introduction

Unit testing is a critical and resource-intensive phase in the software development lifecycle, forming the foundation for building robust software. However, writing high-quality unit tests remains a tedious and time-consuming task for developers [7, 8]. The goal of automated test case generation is to alleviate this burden by generating high-quality test cases that can cover diverse program behaviors and detect faults efficiently. Throughout this paper, we use *robustness* to mean a test

*This work was completed while the authors were studying at Shanghai Jiao Tong University.

Authors’ Contact Information: Pengyu Chang, Shanghai Jiao Tong University, Shanghai, China, pengyuch@andrew.cmu.edu; Yixiong Fang, Shanghai Jiao Tong University, Shanghai, China, yixiongf@andrew.cmu.edu; Silin Chen, Shanghai Jiao Tong University, Shanghai, China, cs12457029646@163.com; Yuling Shi, Shanghai Jiao Tong University, Shanghai, China, yuling.shi@sjtu.edu.cn; Beijun Shen, Shanghai Jiao Tong University, Shanghai, China, bjshen@sjtu.edu.cn; Xiaodong Gu, Shanghai Jiao Tong University, Shanghai, China, xiaodong.gu@sjtu.edu.cn.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/10-ARTISSTA031

<https://doi.org/10.1145/3832122>

suite’s ability to expose diverse fault behaviors, captured by our primary metric, the fault detection rate (FDR).

There have been various approaches for automated test case generation, such as random testing [49], symbolic execution [10, 45, 59], property-based testing [9, 16], and search-based software testing (SBST) [23, 44]. While these traditional methods can achieve substantial code coverage, the resulting tests are often less readable and harder to maintain [21]. As a result, they can increase developer effort for debugging and comprehension, as well as generate too few assertions for effective fault detection.

In recent years, the growing capabilities of LLMs to generate and understand human-readable code [14, 29, 40, 42, 53, 64, 65, 69] have provided new opportunities for automated test case generation. For instance, UTGen [21] integrates LLMs into the SBST process, yielding tests that are both effective and understandable. Similarly, CodaMosa [39] addresses the fitness plateau problem in search-based testing by incorporating LLMs into the test generation process. As a result, CodaMosa outperforms its baseline methods, such as Pynguin [44] and Codex [13], in terms of code coverage. Additionally, HITS [66] demonstrates LLMs’ ability to generate high-coverage tests for complex methods by generating tests slice by slice.

However, much of this work evaluates generated tests primarily using code coverage metrics; fewer studies explicitly optimize for bug-detection capability, particularly robustness against edge cases or boundary conditions. It is widely acknowledged that high coverage does not necessarily equate to strong fault detection [11, 26, 28]. Recent LLM-based studies typically rely on environmental feedback to iteratively improve their test suites. However, most of these methods use only compiler error messages or coverage metrics for prompt refinement [15, 31]. This approach often overlooks logical or semantic faults because coverage metrics only quantify the extent of code execution, not whether the correctness of that execution is rigorously verified. Consequently, a high coverage test suite may still fail to distinguish between correct and incorrect program behaviors.

To address these gaps, we turn to mutation testing (MT), a white-box testing technique that evaluates the ability of a test suite to detect faults. MT injects artificial faults, called *mutants*, into the program by making slight, grammatically correct changes to the code. The test suite is then executed on both the original program and each mutant, with any test that produces a different outcome on a mutant being counted as having “killed” that mutant. The mutation score (MS) is the ratio of killed mutants to the total number of generated mutants.

Building on these insights, we propose **ADVERTEST**, a **Mutation-guided, Adversarial, LLM-driven, Dual-agent** unit test generation framework designed to enhance bug detection capabilities. ADVERTEST pairs a *Test Case Generation Agent* ($\mathcal{T}$), which writes tests to detect bugs, with a *Mutant Generation Agent* ($\mathcal{M}$), which produces mutants that evade $\mathcal{T}$ ’s current suite. The two agents co-evolve through a bidirectional feedback loop guided by mutation score and coverage, exposing each other’s blind spots over successive rounds; we detail the loop in Section 3.7.

We evaluate ADVERTEST on real-world Java projects from Defects4J [37] and GrowingBugs [33–35], and additionally on 12 Python issues from SWE-Rebench [5] to verify cross-language generalization. Compared with state-of-the-art baselines (HITS [66], ChatUniTest [15], EvoSuite [23], and Randoop [49]), ADVERTEST significantly improves fault detection while remaining competitive on line and branch coverage, and the same adversarial loop carries over to Python with 11/12 detection. Ablation studies further isolate the contribution of the adversarial iteration loop and the mutant-aware feedback mechanism; complementary analyses examine the effect of iteration rounds and the choice of backbone LLM.

In summary, our key contributions are as follows:

- We propose ADVERTEST, an adversarial dual-agent framework in which two LLM-driven agents generate mutants and test cases with bidirectional feedback on mutation scores and coverage.
- We evaluate ADVERTEST on real-world projects in both Java (Defects4J [37] and Growing-Bugs [33–35]) and Python (SWE-Rebench [5]), demonstrating an 8.56% gain in fault detection over the strongest LLM baseline and a 50.20% gain over the search-based baseline, while remaining competitive on line and branch coverage.
- We make ADVERTEST publicly available online [12] to facilitate replication and future extensions.

2 Related Works

2.1 Automated Test Case Generation

Automated unit test generation has progressed from early heuristic approaches to advanced AI-driven methods. Traditional techniques such as feedback-directed random testing (e.g., Randoop [49]) and search-based tools (e.g., EvoSuite [23]) achieve high code coverage but often produce tests that are hard to maintain or understand. To improve quality, researchers reframed test generation as a code synthesis task solvable with machine learning. For example, Tufano *et al.* trained a transformer model on code–test pairs (AthenaTest) to automatically generate JUnit tests for a given method [61]. Subsequent neural approaches introduced refinements: A3Test added assertion knowledge and naming consistency checks to improve correctness [1], and other systems (e.g., ConTest, TeCo, CAT-LM) enhanced semantic understanding and output readability [46, 50, 62].

The emergence of LLMs has further accelerated progress. Empirical studies showed that modern code-generating LLMs (e.g., Codex or GPT-3.5) can produce unit tests in a human-like style [55], but they struggled to achieve high coverage on complex code and often introduced “test smells” (redundant or trivial tests) [56]. To better harness LLMs, researchers have integrated these models with program analysis and feedback. For instance, tools like TESTPILOT and CHATUNITEST pair LLM-based generation with static analysis and verification loops to produce valid, high-coverage tests [15, 52]. Hybrid strategies have also emerged: for example, CODAMOSA invokes an LLM when a search-based test generator hits a coverage plateau, generating tests for hard-to-cover functionality [39]. Additionally, some frameworks use an iterative loop where an LLM refines its tests based on feedback from prior test runs (e.g., coverage gaps or errors) [31, 54]. COVERUP uses coverage rate as a feedback and achieves a higher coverage than CODAMOSA on most modules [3]. These LLM-driven techniques are yielding test suites that are not only more readable but also more effective at finding bugs, with substantial gains in coverage and fault detection reported in both research and industry [2].

Unlike prior LLM-based generators that use one-way feedback (e.g., compiler errors or coverage gaps) to refine tests, ADVERTEST introduces a second LLM that creates context-aware mutants and engages the test generator in an adversarial loop. This bidirectional loop, guided by mutation-score and coverage feedback, pushes each agent to close the other’s blind spots and yields more robust fault detection.

Co-evolutionary and adversarial dynamics between tests and program variants pre-date the LLM era and inspire our design. Arcuri and Yao [4] proposed a co-evolutionary search loop in which a program and its test suite alternately evolve to overcome each other. Closer to our setting, *Code Defenders* [24] gamifies mutation testing for education by assigning human students to opposing mutant-writing and test-writing teams, mirroring the role split between Agents $\mathcal{M}$ and

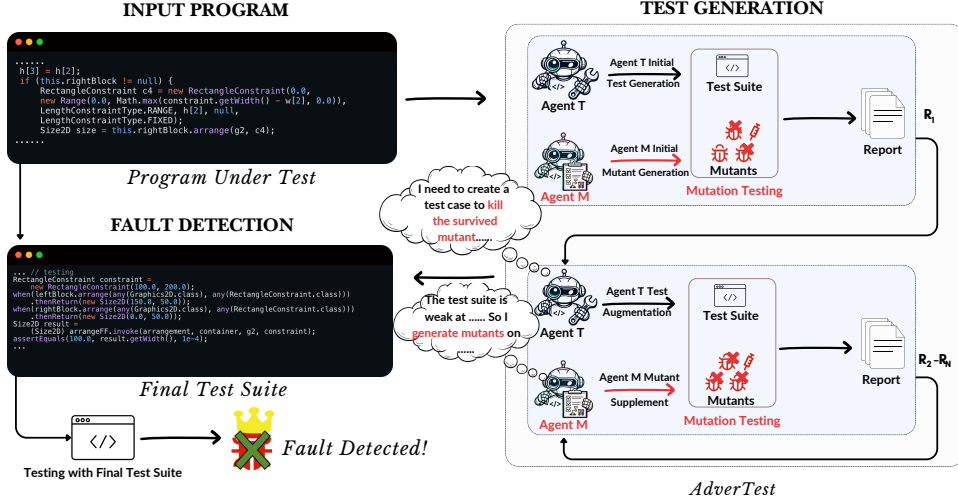


Fig. 1. Overview of ADVERTest. Agents T and M alternately generate tests and create mutants, guided by coverage and mutation-score feedback.

$\mathcal{T}$. ADVERTest operationalizes this adversarial structure with LLM-driven agents, replacing fixed operator sets and human players with context-aware, feedback-driven generation.

2.2 Mutation Testing

Mutation testing evaluates a test suite’s rigor by seeding artificial faults (mutants) into the program and checking if the tests detect them [22]. In the classical approach, developers apply simple code modifications (mutation operators) to produce mutants, then run the suite on each mutant; the fraction of mutants causing test failures (the mutation score) indicates the suite’s fault-detection effectiveness [17, 38].

MT has long been used in traditional automated test generation workflows; for example, EvoSuite [23] applies mutation testing to synthesize assertions. In LLM-based methods, the effects of MT have not yet been thoroughly explored. MuTAP [18] was the first to explore the generation of LLM-based test cases with mutation testing. MuTAP integrates surviving mutants into LLM prompts to improve fault detection, but was evaluated solely on HumanEval [13] and Refactory [30], a dataset of student submitted buggy programs, not on industrial scale projects. Despite this narrow scope, MuTAP achieved notable gains in both mutation score and bug detection rate. Recently, Harman et al. utilized LLM-generated mutants in test generation, but it is not adversarial and evolutionary [27]. Barboni et al. employ an ML model to evaluate the usefulness of each surviving mutant, then prompt them to LLM to generate test cases for smart contracts [6].

Building on this idea, large pre-trained models have been employed to generate mutants without manual rule design. For instance, μ BERT repurposes a Transformer-based code model (CodeBERT) to suggest likely mutations by predicting masked tokens in code. LLMORPHEUS prompts an LLM to inject diverse bugs into code [20, 60].

Studies have shown that LLM-generated mutants are more diverse and effective at revealing bugs than those from conventional tools: Wang *et al.* report that GPT-4 mutants improved real

fault detection by nearly 30% over the best rule-based approach in a benchmark evaluation [63]. Similarly, LLM-created mutants often mimic real vulnerabilities, breaking the same test cases as the actual faults [25].

ADVERTEST further utilizes LLM generated mutants in the test case generating process. By replacing fixed mutation operators with an LLM and alternating test and mutant reinforcement, ADVERTEST is one of the first frameworks to couple adversarial LLM test generation with LLM mutant generation, outperforming both search-based tools and earlier LLM methods on real-world defects.

3 Methodology

In this section, we introduce **ADVERTEST**, an adversarial mutation-guided unit test generation framework in detail.

3.1 Framework Overview

Figure 1 illustrates the overall framework of ADVERTEST, which consists of an adversarial loop between a *Test Case Generation Agent* ($\mathcal{T}$) and a *Mutant Generation Agent* ($\mathcal{M}$). These agents iteratively refine the test suite and generate increasingly challenging mutants. The specific mechanisms of these components and the iterative process are detailed in Sections 3.2-3.7.

3.2 Initial Test Suite Generation

Given a target program P , the agent $\mathcal{T}$ is instructed to generate an initial test suite T . The generation prompt consists of three components: (1) a high-level instruction (e.g., “generate unit tests for the following program”), (2) the complete source code of the program under test, and (3) relevant contextual information, such as surrounding method signatures and class constructors. The full prompt template is designed as follows:

Prompt Template for Initial Test Suite Generation

[Instruction]

You are an expert Java developer and software tester. Your task is to generate full JUnit test methods for a given Java method inside a Java class. Follow these steps to ensure comprehensive and effective test coverage:

(1) **Analyze the Java Method:**

...

(2) **Design Test Cases:**

...

(3) **Implement the Test Method:**

...

[Example]

(An Example of a Java Method and corresponding Test Case)

[Task Inputs]

Given the following Java method, generate a complete JUnit test method that thoroughly tests the method. Utilize your reasoning ability to ensure that all possible scenarios and edge cases are considered.

Input Java Method ({method_name}):

method_body: {method_code}

Class Context:

- **Other Class Variables:** {class_variables}
- **Other Methods in the Class (no method body shown):** {method_info}
- **Constructors of the class object:** {Constructors}

[Guidelines]

(Specific Guidelines for generation, including Java and JUnit version and output format)

As an example, we show the prompt template for Agent $\mathcal{T}$'s initial test case generation. In the Instruction part, we assign the agent's specific roles, Java developer and software testing engineer, and give an initial description of their tasks. Then, we apply a chain-of-thought (CoT)[67] inspired approach to outline the procedures and guidelines that they should follow. In the Example section, we provide a few examples for a model to learn how to generate test cases. In the Task Inputs section, we provide a detailed task description along with all the necessary contextual information. Finally, the Guidelines section offers targeted guidance, including the exact versions of any required software packages and other relevant details. Our methodology follows an iterative prompt engineering process, where each prompt is tested and refined based on observed results. The targeted guidance in the Guidelines section also incorporates common failure modes previously exhibited during the process, which we manually identified and embedded in the prompt, to significantly reduce the likelihood of repeated errors.

Following previous works of ChatUniTest [15] and HITS [66], each initial test case undergoes a repair process to ensure that only syntactically valid and runnable tests are included in the generated test suite T . The raw test cases are compiled and executed against the original program P . If the test fails to compile or execute, we apply 6 deterministic rules to fix them (e.g., fixing missing semicolons, balancing braces).

Having applied all rules, the initial test is recompiled and re-executed. The process stops at the first successful revision.

If all rule-based attempts fail, we invoke an LLM-guided repair process for up to K rounds. In each round, an LLM is instructed with a bug fix prompt, consisting of the last candidate test and its corresponding compilation or runtime error messages. The LLM returns a revised version, which is again compiled and executed. The first syntactically correct and passing revision is accepted.

Following previous works [15, 66], we set $K = 10$. If a test cannot be repaired successfully within the allocated attempts, it is discarded.

It is worth noting that while all tests in T are compilable, they may still include runtime failures (e.g., assertion errors) by design, as these are often indicative of bugs in the program under test. Such failure-exposing tests are retained, as they are valuable for driving mutation testing and bug detection in subsequent phases.

3.3 Initial Mutant Generation

While Agent $\mathcal{T}$ generates an initial test suite, Agent $\mathcal{M}$ works in parallel to generate an initial set of mutants M for the program under test. The goal is to generate diverse and behaviorally meaningful mutants that are both syntactically valid and semantically different from the original program.

The mutant generation process is conducted in a prompt-driven manner. Specifically, we instruct the LLM with a meticulously designed prompt comprising three components: (1) A natural language instruction of the mutation task, (2) The complete code context of the program P , and (3) A set of mutation examples formatted as JSON objects, each illustrating a valid single-line mutation. The few-shot examples are drawn from a previous work by Wang *et al.* [63], which curated mutation examples from the QuixBugs [41] benchmark, which was different from our evaluation dataset while being representative of bugs.

To further promote mutation diversity and correctness, we adopt prior research on prompt-based mutation [60, 63]. Specifically, we enforce a **single-line modification** constraint. This design choice

is grounded in the two fundamental hypotheses of mutation testing: the *Competent Programmer Hypothesis* and the *Coupling Effect*. The latter asserts that “test data that distinguishes all programs differing from a correct one by only simple errors is so sensitive that it also distinguishes more complex errors” [22, 47]. By restricting mutants to single-line changes, we focus on these relatively simple errors, ensuring the generated test suite is sensitive enough to detect complex faults while minimizing the generation of uncompileable code common in unconstrained LLM generation [63].

Accordingly, we provide the following constraints in the instruction:

- Only one mutation is allowed per mutant.
- Each mutation must modify exactly one line of code.
- Redundant or meaningless mutations (e.g., altering comments or whitespace) are disallowed.
- Output format is strictly specified to enable parsing and integration into the mutation testing infrastructure.
- Previously generated mutants must not be repeated.

Prompt Template for Initial Mutant Generation

[Instruction]

Below is a code snippet from a Java project. Your task is to generate {MUT_NUM} mutants for this code. (Note: A mutant refers to a syntactically valid variant with a subtle alteration used for software testing.)

[Input Code]

{code}

[Few-Shot Examples]

Refer to the following format for the expected output logic:

```
{
  "id": "1",
  "precode": "return depth==0;",
  "aftercode": "return true;",
  "line_number": 42
}
```

[Requirements]

- (1) Provide the generated mutants directly in the specified output format.
- (2) **Single Line Modification:** Ensure each mutant affects **only one line**. Pay strict attention to line breaks; statements may need to be split.
- (3) **Output Format:** Adhere strictly to the JSON format shown above.
- (4) **Uniqueness:** Do not generate duplicate mutants.
- (5) **No meaningless mutants:** Do not generate mutants that result in identical execution behavior to the original.

3.4 Mutation Testing

Once the initial test suite T and the mutation set M have been generated, we perform mutation testing on the program under test P following Algorithm 1.

Let P denote the program under test, M be the set of mutants created by Agent $\mathcal{M}$, and T the current test suite produced by Agent $\mathcal{T}$. We treat P as the reference (“bug-free”) program throughout the generation loop. Agents $\mathcal{T}$ and $\mathcal{M}$ receive only P as input; any buggy variant of P , if available, is used solely *outside* the loop to score fault detection. For each mutant $m \in M$, the compile step is the sole execution gate: m is added to the valid set M_v if and only if it compiles within the sandbox (Algorithm 1, Line 4). Mutants that fail to compile or exceed a wall-clock timeout are discarded as invalid and excluded from all subsequent metrics. Valid mutants are then executed

Algorithm 1: Mutation Testing**Input:** Program P , test suite T , mutants M **Output:** Surviving mutants M_s , coverage C , mutation score S , valid mutants M_v

```

1  $M_s \leftarrow \{\}$ ;
2 foreach  $m \in M$  do
3   if  $m$  compiles then
4      $M_v \leftarrow M_v \cup \{m\}$ ;
5      $\Delta \leftarrow \text{RUNTESTS}(T, m)$ ;
6     if  $\Delta = \emptyset$  then
7        $M_s \leftarrow M_s \cup \{m\}$ ;
8     end
9   end
10 end
11  $C \leftarrow \text{COMPUTECOVERAGE}(T, P)$ ;
12  $S \leftarrow (|M_v| - |M_s|) / |M_v|$ ;
13 return  $(M_s, C, S, M_v)$ ;
```

exactly *once* as part of the suite-evaluation step $\text{RUNTESTS}(T, m)$ in Line 5; there is no separate “validate-then-rerun” phase.

A mutant is considered **survived** if it exhibits no behavioral difference from the original program. Concretely, let $\text{Fail}(T, x)$ denote the set of test cases in T that fail when executed on program x ; a mutant m survives if:

$$\Delta = \text{Fail}(T, m) \setminus \text{Fail}(T, P) = \emptyset$$

The set subtraction is retained even though P is bug-free by construction, because LLM-generated tests can be flaky or contain hallucinated assertions that fail on P itself; subtracting $\text{Fail}(T, P)$ ensures that only failures *caused by the mutation* count toward killing the mutant. All surviving mutants are added to the surviving set M_s . Conversely, remaining valid mutants that are detected by at least one test case are considered **killed**.

This mutation testing process produces two key feedback signals: (1) the *mutation score* $S = (|M_v| - |M_s|) / |M_v|$, which quantifies how many mutants remain undetected by the current test suite, and (2) the structural *coverage* C , calculated as $C = \frac{|E_{\text{covered}}|}{|E_{\text{total}}|}$, which captures the ratio of executed structural elements (e.g., lines, branches) to the total number of coverable elements.

These metrics jointly drive the adversarial interaction between the two agents:

- Agent $\mathcal{T}$ leverages the mutation score S to refine or regenerate test cases against surviving mutants M_s ;
- Agent $\mathcal{M}$ leverages both S and C to craft new mutants in structurally weak or under-tested regions of the program.

Overall, mutation testing acts as the central feedback mechanism in ADVERTEST, enabling bidirectional improvement: it strengthens Agent $\mathcal{T}$ ’s test generation capabilities while guiding Agent $\mathcal{M}$ to synthesize more challenging mutants. This adversarial loop drives the system toward progressively more robust and comprehensive test suites.

3.5 Test Suite Augmentation

The mutation testing produces a set of surviving mutants M_s , where each mutant $m \in M_s$ represents a behavioral variation that the current test suite does not detect. These surviving mutants effectively expose the blind spots of T .

To fill these blind spots, we augment T by generating new tests aimed at “killing” each surviving mutant. Specifically, for each $m \in M_s$, we construct a mutant-aware prompt that summarizes the mutant in natural language (e.g. “original line: return $x+y$; mutated to return $x-y$;”), and provide this prompt to Agent $\mathcal{T}$. The agent is instructed to generate a test case that fails on the mutant variant P_m while passing on the original program P .

All generated test cases undergo the same test-repair loop as described in Section 3.2. This ensures that only syntax-correct, compilable, and behaviorally valid test cases are included in the augmented suite. Once repaired and validated, the new test is added to T . This process is repeated for each surviving mutant, gradually evolving the test suite toward higher fault-detection capability and robustness.

Importantly, we do *not* immediately re-evaluate each new test against its associated mutation after generation. Prior work [57] has shown that while immediate feedback can increase mutant detection rates, it comes at a substantial cost: up to $\times 7.29$ higher LLM API token usage and significantly increased mutation testing time. Instead, we defer evaluation of newly added tests until the next augmentation cycle. Mutants that still survive will be re-targeted in subsequent rounds by Agent $\mathcal{T}$, allowing us to eventually detect most mutants without incurring excessive computational and financial (the API cost) overhead.

Following this augmentation process, the resulting test suite, T^+ , is free from syntax and compilation errors. This augmented suite is then either passed to the next iteration of the adversarial loop or returned as the final output if termination conditions are met.

3.6 Mutant Augmentation

The agent $\mathcal{M}$ uses the two key feedback signals obtained from mutation testing: the set of surviving mutants M_s and the structural coverage map C to augment its mutant pool in two complementary directions.

1. *Augmentation by Uncovered Code.* Structural coverage C allows us to extract a set of uncovered lines L_u , which are code regions that remain untested by any case in T . These lines present latent risks, as they may contain faults that the current testing process has yet to examine. Agent $\mathcal{M}$ proactively attacks these uncovered lines by generating new mutants at those locations, even in the absence of prior mutant feedback. This strategy forces the test suite to interact with previously ignored control paths and execution traces, thereby improving both code coverage and fault exposure.

2. *Augmentation by Surviving Mutants.* Generating new mutants solely based on structural coverage is often insufficient to provide Agent $\mathcal{T}$ with actionable feedback. Even when a line of code is covered, it may still conceal faults. For example, when a test case executes a statement without asserting its effects, behavioral deviations remain undetected. As coverage increases, the available space for purely coverage-driven mutation gradually diminishes, limiting the effectiveness of further exploration.

In this regime, surviving mutants provide a valuable signal. Mutants in M_s indicate program locations where the current test suite T fails to detect behavioral divergence from the original program. These locations reveal structural weaknesses in the test suite that are not captured by coverage metrics alone. To exploit this signal, we first group surviving mutants by the specific lines of code they modify, thereby reducing redundancy and focusing mutation efforts on under-tested locations. For each group, we construct prompts that instruct the LLM to generate new and diverse mutations on the same line, while varying logic, constants, or operators. This strategy allows Agent $\mathcal{M}$ to systematically explore a richer space of plausible faults rooted in a shared structural vulnerability, rather than repeatedly mutating already well-tested code.

By integrating mutation generation driven by both surviving mutants and coverage gaps, this augmentation mechanism ensures that the mutant process remains both adaptive (responding to observed weaknesses in the test suite) and exploratory (continuing to probe untested or weakly tested regions). This balance enables a more effective adversarial co-evolution between Agent $\mathcal{T}$ and Agent $\mathcal{M}$, ultimately leading to more robust and discriminative test suites.

3.7 Adversarial Iteration Loop

The two agents interact through a structured adversarial loop (Algorithm 2) that runs for a predefined number of rounds N . At each iteration, the agents alternate their actions to progressively improve the test suite quality.

- **Test Suite Augmentation:** Agent $\mathcal{T}$ acts first (Line 5) by generating new tests specifically designed to kill the surviving mutants (M_s) identified in the evaluation step.
- **Mutant Augmentation:** Agent $\mathcal{M}$ responds (Line 6) by synthesizing additional mutants to exploit blind spots in the code that remain uncovered or under-tested.

Once the loop completes ($i = N$), the final action within the loop corresponds to Agent $\mathcal{M}$. To prevent the process from ending with a batch of unchecked mutants, we perform a final round of test suite augmentation (Line 9). This ensures that Agent $\mathcal{T}$ always has the final move, guaranteeing that the returned test suite T has responded to the most recent adversarial inputs.

Algorithm 2: Main adversarial loop of ADVERTEST

Input: Program P , prompts π_0, μ_0, λ_0 , max rounds N

Output: Final test suite T

```

1  $T \leftarrow \text{GENTEST}(P, \pi_0, \lambda_0)$ ;           // Agent  $\mathcal{T}$  plays initial move
2  $M \leftarrow \text{GENMUTANT}(P, \mu_0)$ ;         // Agent  $\mathcal{M}$  plays initial move
3 for  $i \leftarrow 1$  to  $N$  do
4    $(M_s, C, S, M_v) \leftarrow \text{MUTATIONTESTING}(P, T, M)$ ;           // Evaluate the state of the loop
   // Agent  $\mathcal{T}$  turn: Eliminate surviving mutants
5    $T \leftarrow \text{ENHANCETESTCASEBYMUTANTS}(T, M_s, \pi_0, \lambda_0)$ ;
   // Agent  $\mathcal{M}$  turn: Exploit blind spots
6    $M \leftarrow \text{ENHANCEMUTANTSBYFEEDBACK}(M, T, C, \mu_0)$ ;
7 end
   // Ensure  $\mathcal{T}$  responds to  $\mathcal{M}$ 's last move
8  $(M_s, C, S, M_v) \leftarrow \text{MUTATIONTESTING}(P, T, M)$ ;
9  $T \leftarrow \text{ENHANCETESTCASEBYMUTANTS}(T, M_s, \pi_0, \lambda_0)$ ;
10 return  $T$ ;
```

4 Experimental Setup

We evaluate ADVERTEST by addressing the following research questions:

- **RQ1:** How effective is ADVERTEST in generating tests in real-world projects?
- **RQ2:** What is the individual contribution of each component of ADVERTEST to overall effectiveness?
- **RQ3:** How do iterative rounds affect ADVERTEST's effectiveness?
- **RQ4:** Does ADVERTEST generalize beyond Defects4J to other languages and ecosystems?

4.1 Datasets

We evaluate ADVERTEST on Defects4J [37] and GrowingBugs [33–35], two datasets that provide authentic and reproducible defects in industrial-scale Java code bases. With a total of 20 different projects, 247 different bugs, 727 methods under test. This scale significantly surpasses the evaluations of previous work, such as HITS [66] (120 methods) and ChatUniTest [15] (264 methods).

Defects4J. Defects4J is a widely adopted benchmark of real, reproducible faults in open source Java projects [37]. In version 2.1.0, it comprises 835 more bugs drawn from 17 different and high quality projects, each paired with a corresponding fixed version and a comprehensive test suite. Each defect entry includes metadata such as affected files, trigger tests, and developer patches, allowing repeatable fault detection and repair experiments. In this experiment, we used 200 randomly sampled defects from all 17 projects. We also leverage Defects4J framework throughout our experiment and evaluation.

GrowingBugs. GrowingBugs is an extensible repository of real faults in open source Java projects built on top of the Defects4J infrastructure [37]. GrowingBugs automatically filters out non-functional changes from commit histories using the BugBuilder tool, enabling continuous expansion of the dataset without human intervention [33, 34]. Previous studies have shown that patches extracted by the BugBuilder preserve the naturalness of real bugs and support robust empirical evaluations of testing and repair techniques [35]. To mitigate potential data leakage, we specifically selected all 3 projects that were introduced into the GrowingBugs benchmark after the knowledge cutoff date of the LLM we use for that experiment: commons-dbcp (Dhcp), commons-beanutils (Beanutils), and commons-fileupload (Fileupload). This subset comprises a total of 47 bugs and 89 methods under test. We caution that “recently added to GrowingBugs” applies to the benchmark inclusion date, not necessarily to the individual bug introduction dates: some defects in these projects pre-date the LLM training cutoff and could in principle have been observed during pre-training. To further address this residual leakage concern, Section 5.4 reports an additional generalization study on 12 Python bugs drawn from real GitHub issues post-dating the model cutoff.

4.2 Baselines

We compare ADVERTEST against four baselines, two traditional and two LLM-driven: *Randoop* [49], a feedback-directed random Java test generator; *EvoSuite* [23], a search-based tool that evolves JUnit suites toward high line and branch coverage; *ChatUniTest* [15], an LLM-driven generator that supplies the focal method plus rule-extracted context and uses error-driven test repair (its coverage drops on complex methods [66]); and *HITS* [66], an LLM-driven generator that decomposes complex methods into semantically coherent slices and generates tests per slice, achieving the highest LLM-baseline coverage on complex methods. We configure EvoSuite and Randoop with a 500 seconds per-bug budget, comparable to ADVERTEST’s runtime on Defects4J.

Both LLM baselines use DeepSeek-v3.2 and GPT-OSS-120B [19, 48] (we restrict GrowingBugs to DeepSeek-v3, whose cutoff predates the dataset [66]). We adapt both for Defects4J: prompts target JUnit 4 instead of JUnit 5, and tests are executed with the Defects4J command. Other LLM- and mutation-guided systems are not directly comparable in the context of industrial Java projects. MuTAP [18] targets Python and is evaluated only on student-program benchmarks; Meta’s mutation-guided system [27] is closed-source; and Alchemist [6] focuses on Solidity smart contracts.

4.3 Metrics

We employ three metrics in our experiments.

(1) Fault Detection Rate (FDR): As our study focuses on the effectiveness of test generation in detecting real faults, the fault detection rate serves as our primary metric [51]. Let $\mathcal{F} = \{f_1, f_2, \dots, f_N\}$

be a set of N known defects, where each f_i has a fixed (reference) version P_i^{fix} and a buggy version P_i^{bug} , let T_i be the test suite generated for f_i , and let $\text{Fail}(T, P)$ denote the tests in T that fail on program P . The detection indicator is defined as:

$$\text{Detect}(T_i, f_i) = \begin{cases} 1, & \text{if } \text{Fail}(T_i, P_i^{bug}) \setminus \text{Fail}(T_i, P_i^{fix}) \neq \emptyset \\ 0, & \text{otherwise} \end{cases}$$

Then, FDR is computed as:

$$\text{FDR} = \frac{1}{N} \sum_{i=1}^N \text{Detect}(T_i, f_i)$$

All methods, including the baselines, generate T_i on the fixed version P_i^{fix} ; the buggy version P_i^{bug} is used only to score detection by replaying the suite. Section 6 discusses this reference-version setup.

(2) Coverage: In addition to fault detection, we also report the coverage of each generated suite (measured with Cobertura[58]) in accordance with previous works [15, 66]. We report both *line* and *branch coverage*. *Line coverage* measures the percentage of lines that have been executed by the test cases. *Branch coverage* measures the percentage of branches (decision points) in the source code that have been executed during the testing process [36].

(3) Cost: We also evaluate the API token cost of LLM-based methods. For API access, we use the official DeepSeek platform for DeepSeek models and Tinker for GPT-OSS models. We track token usage throughout the entire generation process of each method and report the average cost per method on each dataset.

4.4 Parameter Configuration

We run our method on multiple backbone LLMs, including DeepSeek-v3.2, GPT-OSS-120B for both agents and LLM-based baselines. We choose these models for their balance between strong coding ability and low cost. To minimize randomness in test generation, we set temperature=0 as suggested for all LLM-based test generation methods [43]. All tools and generated tests are built and executed under Java 8 with JUnit 4 and Mockito 4.11 to ensure compatibility with Defects4J. We limit the adversarial loop to a maximum of 5 iterations. Excluding the initial generation process, this comprises a combined total of 5 actions between the agents, beginning and ending with Agent $\mathcal{T}$. Agent $\mathcal{M}$'s per-invocation mutant budget is set adaptively to $\max(2 \text{ line_num}, 50)$, scaling with method length. To measure coverage, all suites are instrumented and measured with Cobertura [58] using its default configuration provided by Defects4J framework[37].

5 Results and Analysis

5.1 RQ1: Effectiveness

We evaluate the effectiveness of ADVERTest by comparing its FDR and coverage against traditional and LLM-based baselines. The results are summarized in Table 1.

Fault Detection Capability. ADVERTest consistently achieves high fault detection rates across all datasets. On the Defects4J dataset, ADVERTest (using DeepSeek V3.2) attains an FDR of **66.63%**. This represents a relative improvement of approximately **8.6%** over the state-of-the-art LLM-based method, HITS (61.38%), and a substantial **50.20%** improvement over the traditional search-based tool, EvoSuite (44.36%). A similar trend is observed on the GrowingBugs dataset, where ADVERTest achieves **65.96%** FDR compared to 61.70% for HITS and 40.43% for EvoSuite. This result is particularly significant, as GrowingBugs contains defects introduced more recently, serving as a rigorous test for data leakage and overfitting.

Table 1. Comparison of Fault Detection Rate (FDR), Coverage, and Cost on Defects4J and GrowingBugs Datasets. Best results for each model are highlighted in **bold**. Second best results are underlined.

Method	Defects4J				GrowingBugs			
	FDR	Line Cov.	Branch Cov.	Cost	FDR	Line Cov.	Branch Cov.	Cost
<i>Traditional Approaches</i>								
Randoop	30.21%	38.39%	28.99%	–	23.40%	38.16%	27.34%	–
EvoSuite	44.36%	69.45%	61.54%	–	40.43%	67.86%	64.37%	–
<i>LLM-Based Approaches</i>								
GPT-OSS-120B								
ChatUniTest	21.56%	18.89%	17.83%	\$0.178	–	–	–	–
HITS	33.93%	28.33%	25.97%	\$1.096	–	–	–	–
AdverTest (Ours)	57.05%	60.50%	57.40%	<u>\$0.553</u>	–	–	–	–
DeepSeek								
ChatUniTest	13.52%	14.38%	14.15%	\$0.113	31.91%	32.07%	34.79%	\$0.089
HITS	<u>61.38%</u>	<u>60.13%</u>	<u>52.40%</u>	\$0.411	<u>61.70%</u>	77.99%	<u>66.02%</u>	\$0.407
AdverTest (Ours)	66.63%	62.26%	57.06%	<u>\$0.270</u>	65.96%	<u>74.54%</u>	70.53%	<u>\$0.245</u>

Coverage. In terms of coverage, EvoSuite achieves the highest line and branch coverage on Defects4J under the matched 500 s budget, with ADVERTEST a close second on both metrics. This higher coverage, however, does not translate into higher fault detection: EvoSuite’s FDR (44.36%) still trails ADVERTEST’s 66.63% by a wide margin. EvoSuite primarily optimizes for code execution (coverage and weak mutation), often generating tests that reach a faulty line without propagating the error to an observable output [23]. In contrast, ADVERTEST integrates strong mutation directly into the loop: surviving mutants explicitly guide the LLM to generate tests that distinguish the mutant’s behavior from the original code, ensuring the generated tests not only execute the code but also rigorously expose semantic faults.

On GrowingBugs, HITS leads line coverage thanks to its slicing-based method, while ADVERTEST leads branch coverage and FDR. Unlike HITS [66], ADVERTEST does not focus solely on high-coverage test cases.

Robustness Across Foundation Models. As shown in Table 1, ADVERTEST demonstrates superior robustness across both LLMs. Even with the weaker GPT-OSS-120B model, ADVERTEST maintains a high FDR of **57.05%** whereas the FDR of HITS drops from 61.38% to 33.93%. This suggests that the adversarial feedback loop effectively compensates for the weaker reasoning capabilities of smaller models, whereas HITS’s slicing method relies heavily on the raw capability of the foundation model.

Cost analysis. Among LLM baselines (Table 1, cost columns), ChatUniTest incurs the lowest absolute API cost but at the expense of test quality. Against the strongest baseline HITS, ADVERTEST reduces per-method API cost by **34.3%** on Defects4J w/ DeepSeek-v3.2 (\$0.270 vs. \$0.411), **49.5%** on Defects4J w/ GPT-OSS-120B (\$0.553 vs. \$1.096), and **39.8%** on GrowingBugs (\$0.245 vs. \$0.407). Absolute prices depend on platform pricing policies, but the relative advantage holds across all settings.

Runtime. Per-method wall-clock time on Defects4J: EvoSuite and Randoop use the matched 500 s budget; among LLM-based methods, ChatUniTest averages 122.7 s, ADVERTEST 484.95 s, and HITS 878.63 s. ADVERTEST is therefore roughly 1.8× faster than HITS while detecting more bugs, and runs within the same time envelope as the search-based baselines.

Statistical Significance. A McNemar’s test (appropriate for matched-paired binary detection outcomes) against the strongest baseline HITS, combining both DeepSeek V3.2 and GPT-OSS-120B results on Defects4J, yields **95** bugs that ADVERTEST detects but HITS misses versus **57** the other way, with $p = \mathbf{0.00257}$. We therefore reject the null hypothesis at $p < 0.01$ and conclude that the FDR gain is statistically significant.

Mutant Quality. On Defects4J, Agent $\mathcal{M}$ ’s output achieves a **90.3%** compile rate, **36.2%** survival rate among valid mutants, and an equivalent-mutant rate of **3.45%**—comparable to the **2.1%** reported for the MAJOR mutation tool [63].

Answer to RQ1. ADVERTEST significantly outperforms both search-based and LLM-based baselines, improving fault detection rates by up to 50.20% over EvoSuite and 8.6% over HITS on Defects4J, with consistent generalizability on GrowingBugs. Our method also proves cost-effectiveness by reducing API costs by more than 34.3% compared to HITS. Furthermore, the framework proves highly robust, leveraging the adversarial loop to maintain high effectiveness even when utilizing less capable foundation models.

5.2 RQ2: Ablation Study

To isolate the contribution of each core component in our framework, we conduct an ablation study on a subset of Defects4J dataset with GPT-OSS-120B as the base LLM. This subset contains a total of 50 bugs randomly sub-sampled from the same 200-bug Defects4J set used in RQ1, comprising 129 methods under test from all 17 Defects4J projects. The sampling is chosen to keep the per-variant compute cost tractable; we discuss the resulting representativeness threat in Section 6. Specifically, we evaluate the impact of four critical components:

(1) **Adversarial Iterative Loop:** We disable the iterative co-evolution process between Agent $\mathcal{T}$ and $\mathcal{M}$, limiting the system to a single round of initial LLM-based test generation without any adversarial feedback or augmentation; and

(2) **Surviving Mutant Feedback:** We remove the fine-grained feedback mechanism introduced in Section 3.5, which informs the test generator of the exact nature of surviving mutants. In this setting, the LLM is aware that certain mutants remain undetected but receives no details about their specific locations or semantics. It must therefore generate additional tests without targeted guidance.

(3) **Coverage-only Feedback:** We replace all mutation-based signals with coverage information alone. Agent $\mathcal{T}$ receives the uncovered source lines (with their code) and the affected method bodies in place of any surviving-mutant descriptions. This variant isolates the contribution of mutation-based feedback over a pure coverage-driven signal.

(4) **Classical Mutation Substitute:** We replace Agent $\mathcal{M}$ with the MAJOR mutation tool [38] while leaving every other component of the loop unchanged. Only the source of mutants changes from LLM-generated to rule-based; the feedback channel, the iteration schedule, and Agent $\mathcal{T}$ are identical to the full pipeline. This variant isolates the contribution of LLM-driven mutation over a classical rule-based mutation engine.

Table 2 shows that removing the iterative loop (**w/o Iter**) incurs the largest drop in FDR (51.85%) and coverage (22.17% line / 30.75% branch). Omitting mutant-aware prompting (**w/o Mut**) also degrades performance substantially, confirming that providing the LLM with concrete mutation details is critical for generating effective tests. However, ‘w/o Mut’ still achieves higher metrics compared to ‘w/o Iter’, that is because we still provide Agent $\mathcal{T}$ chances to improve the test case, even without specific information, knowing surviving mutants exists is a benefit. Also, missing

Table 2. Ablation Study on the Defects4J Dataset

Variant methods	FDR	Coverage	
		Line	Branch
ADVERTEST	54.00	88.17	82.01
w/o Iter	26.00	68.62	56.79
w/o Mut	40.00	77.52	66.22
w/ Cov-only	34.00	80.33	69.72
w/ MAJOR	46.00	85.71	80.12

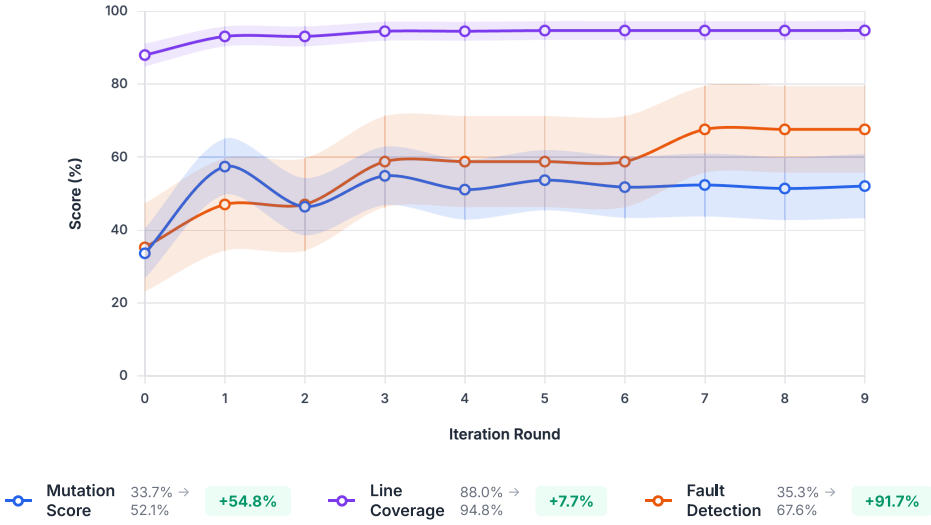


Fig. 2. Mutation Score (MS), Line Coverage (CV), and Fault Detection Rate across Nine Rounds. The shadow indicates the standard deviation.

mutant information will lower the mutation score and the ability to 'kill' surviving mutants, which will result in more test case enhancement rounds.

Replacing mutation feedback with coverage-only signals (**w/ Cov-only**) drops FDR to 34.00%, which is lower than **w/o Mut**'s 40.00% despite slightly higher coverage (80.33% line). This suggests that surviving-mutant feedback carries information beyond what coverage signals alone provide. Substituting the LLM mutator with the rule-based MAJOR tool (**w/ MAJOR**) loses 8 pp FDR (from 54.00% to 46.00%) while coverage remains nearly intact (85.71% line), suggesting that the LLM mutator's contribution stems more from qualitative mutant diversity than from reaching new lines.

The results indicate that all four ablation variants degrade performance: ablating or substituting any single component reduces FDR by 14.81%–51.85% and coverage by 2.30%–30.75%.

Answer to RQ2. All four ablation variants confirm ADVERTEST's design: the iterative loop is the single largest driver of FDR; mutation-based feedback contributes information that pure coverage signals cannot capture; and LLM-driven mutation outperforms a classical rule-based mutator while keeping coverage comparable.

5.3 RQ3: Effect of Iterative Rounds

To analyze the impact of iteration rounds on ADVERTEST's performance, we executed the adversarial loop for a total of 9 rounds on the subset defined in Section 5.2, utilizing GPT-OSS-120B as the underlying model. In this setup, a single "round" corresponds to one action by either Agent $\mathcal{T}$ or Agent $\mathcal{M}$.

Figure 2 illustrates the co-evolution of the metrics. Round 0 represents the performance of the initial test suite generated. In subsequent steps, the agents alternate: Agent $\mathcal{T}$ performs test augmentation in odd rounds (1, 3, 5, 7, 9), while Agent $\mathcal{M}$ generates supplementary mutants in even rounds (2, 4, 6, 8).

We observe distinct behaviors across the three metrics:

Line Coverage (Purple): Coverage shows a steady but modest increase, rising from 88.0% to 94.8% (+7.7%). The high initial starting point suggests that modern LLMs are inherently proficient in achieving structural coverage. Consequently, the marginal gains diminish in later rounds as the code becomes saturated.

Mutation Score (Blue): The Mutation Score (MS) displays a sawtooth pattern characteristic of the adversarial process. During Agent $\mathcal{T}$'s turns (odd rounds), MS increases as the test suite is refined to kill existing mutants. Conversely, during Agent $\mathcal{M}$'s turns (even rounds), MS decreases as new mutants are injected to exploit blind spots. Despite these local fluctuations, the global trend is a significant net increase of 54.8%, indicating that the test suite is becoming progressively more robust against semantic faults.

Fault Detection Rate (Orange): The most substantial improvement is observed in the FDR, which surges from 35.3% to 67.6% (+91.7%). Notably, while coverage increases moderately, FDR continues to rise significantly in later iterations (e.g., the jump at Round 3 and 7). This confirms that the mutation guided adversarial loop successfully directs the agents toward corner cases and logical faults that coverage metrics overlook and generate more robust test cases.

Answer to RQ3. ADVERTEST's effectiveness improves markedly with iterative rounds. While coverage gains are incremental, the adversarial interaction drives a substantial increase in Mutation Score and Fault Detection Rate, demonstrating that the iterative loop is essential for detecting complex, real-world faults.

5.4 RQ4: Generalization Beyond Defects4J

To verify that the adversarial loop transfers beyond Java and Defects4J, we re-implemented ADVERTEST for Python and evaluated it on the SWE-Rebench benchmark [5], which is continuously updated from live GitHub issues. We used 12 instances from the SWE-Rebench 2026-02 leaderboard, all of which post-date the LLM training cutoff, providing a stronger data-leakage control than relying on benchmark inclusion dates alone (cf. Section 6).

Setup. The 12 instances span AWS SDKs (bedrock-agentcore-sdk-python, moto), agent frameworks (a2a-python), build systems (conan), NLP pipelines (haystack), document parsing (docling), filesystem mocking (pyfakefs), ORM tooling (advanced-alchemy, pymilvus), Django templating (django-bird), deep-learning libraries (keras), and notebook runtimes (marimo). We ported the full pipeline of Sections 3.2–3.7 to pytest and coverage.py, keeping all algorithmic components. The backbone LLM is GPT-OSS-120B. Detection follows the same criterion as in Section 4.3: the generated suite *detects* the bug iff some test fails on the buggy version but not on the fixed version.

An Additional Mutant Source. In porting ADVERTEST to Python, we observed a recurring failure mode of the two-source mutant augmentation described in Section 3.5. Many covered, bug-relevant

Table 3. Generalization on 12 SWE-Rebench Python instances (GPT-OSS-120B; MS and coverage averaged across instances for the full pipeline). S1: variations of surviving mutants. S2: mutants on uncovered lines. S3: mutants on covered-but-untouched lines.

Pipeline	Detected	FDR	Avg MS	Avg Cov.
S1 + S2	7/12	58.3%	—	—
S1 + S2 + S3	11/12	91.7%	64.2%	93.0%

lines had no mutant attached, because the initial call concentrated mutants on a few operators per line, so `ENHANCETESTCASEBYMUTANTS` never received a survivor at those locations. We therefore enabled a third augmentation source for the Python pipeline. In addition to (S1) variations of surviving mutants and (S2) mutants on uncovered lines, the agent now also generates (S3) mutants on lines that are covered by the current suite but have no mutant associated with them. We treat S3 as an opt-in extension of the methodology and report both pipeline variants below.

Results. Table 3 reports the head-to-head comparison. Enabling the untouched-line source raises FDR from 7/12 (**58.3%**) to **11/12 (91.7%)**; the four additional bugs become detectable once the round-3 `ENHANCETESTCASEBYMUTANTS` step receives survivors generated by S3. Across the 11 detected instances, 6/11 (54.5%) require at least one enhancement round to surface, and 2/11 (18.2%) are detected only at round 3 or later, where the alternation between S3-driven mutant injection and Agent $\mathcal{T}$'s test refinement is essential.

Why S3 Helps. S3 attaches mutants to covered lines that carry no operator-style mutant, surfacing survivors that drive Agent $\mathcal{T}$ to probe their semantics. For example, in `deepset-ai/haystack#10547`, mutating an `except TypeError` clause to `except BaseException` forced Agent $\mathcal{T}$ to write a test that probes the same semantic boundary as the real defect. We note that this hint is available only because generation operates on the fixed version, which retains the `except` clause that the buggy code omits; we discuss this reference-version setup and its implications in Section 6. The one undetected instance (`getmoto/moto#9727`) reached **83.8%** coverage yet went undetected, a reminder that high coverage alone does not guarantee fault detection.

Leakage Controls. Throughout the loop, the LLM receives only the source code of the *fixed* target function, the file path, and module-level context. It never sees the GitHub issue description, the developer patch, the developer-written tests, the hint text, or the `FAIL_TO_PASS` oracle from SWE-Rebench. The only bug-specific information used by the pipeline is the function-level scope (which method the patch touches), which is the same level of supervision `ADVERTEST` uses on `Defects4J`. The **91.7%** FDR is therefore obtained on bugs whose patches were not in the model's training data and whose oracles the pipeline does not see.

Answer to RQ4. `ADVERTEST` transfers from Java/`Defects4J` to Python/SWE-Rebench, detecting **11/12 (91.7%)** of real-world Python bugs drawn from post-cutoff GitHub issues. This suggests that the gains are not narrowly tied to language, test-tooling, or model memorization.

5.5 Case Study

To demonstrate the effectiveness of `ADVERTEST`, we present a representative case study involving a defect in `jfree-chart` project from **Defects4J** dataset. We selected this case because the fault involves a boundary condition easily overlooked by standard coverage metrics, yet it illustrates

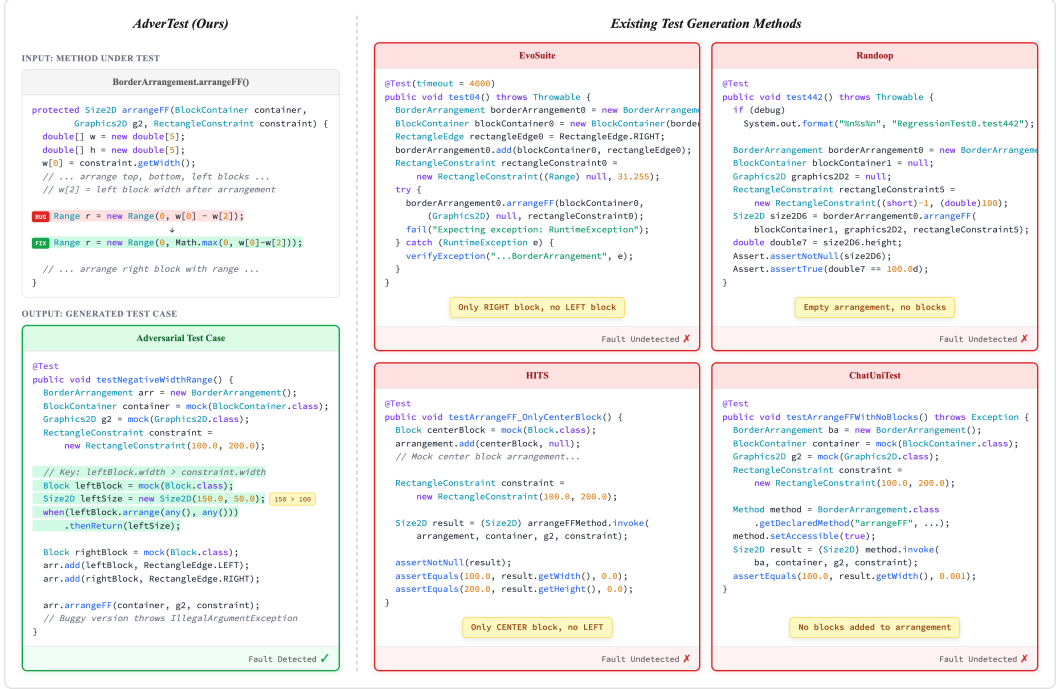


Fig. 3. An Example of Fault Detection Process for arrangeFF.

how the components of ADVERTEST work together. As illustrated in Figure 3, among the evaluated tools only ADVERTEST detected this bug.

The top-left panel of Figure 3 displays the method under test, arrangeFF, along with the fix. This method arranges blocks within a container subject to fixed width and height constraints. The buggy version fails to account for the scenario where the width of the left block exceeds the total constraint width, producing a negative upper bound for the right block's range (calculated as $w[0] - w[2]$) that triggers an `IllegalArgumentException`. This bug is subtle because standard inputs easily cover the faulty statement without triggering the exception, masking the untested critical boundary condition.

At first, Agent $\mathcal{T}$'s initial test generation failed to detect the bug, as the input space was too broad for the LLM to effectively target. Similarly, all baseline methods failed; as shown in figure 3, tools like EvoSuite, Randoop, HITS, and ChatUniTest generated various block layouts, but none produced the specific condition where the left block is wider than the container. However, Agent $\mathcal{M}$ successfully generated 70 mutants, two of which effectively simulated the underlying defect (e.g., by replacing `Math.max` with `Math.min` or removing the function call entirely). These mutants survived the initial test suite. Recall that ADVERTEST generates on the fixed version, so Agent $\mathcal{M}$ produces these mutants by perturbing the corrected code, recreating the historical fault.

In the subsequent test augmentation phase, these surviving mutants provided critical guidance to Agent $\mathcal{T}$. By analyzing the surviving mutants, Agent $\mathcal{T}$ inferred the necessity of a test case where the constraint width is smaller than the left block's width ($w[2]$). Consequently, as shown in the bottom-left panel, Agent $\mathcal{T}$ generated a targeted adversarial test case that satisfied this condition, successfully exposing the fault.

This example illustrates how mutation guidance and the LLM’s semantic understanding work together. First, unlike EvoSuite and Randoop, which produce tests with low readability and maintainability [21], ADVERTEST generates semantically meaningful and readable test code. More importantly, the surviving mutants effectively directed Agent $\mathcal{T}$ to focus on the precise input region required to trigger the fault. Without such guidance, the test search space remains too broad, making the probability of randomly generating a fault-revealing input negligible. Furthermore, this highlights the advantage of LLM-based mutation: unlike traditional operators, LLMs leverage code context to produce compilable, logic-altering mutations—such as changing `Math.max` to `Math.min`—that drive deeper testing.

6 Threats to Validity

We organize threats to validity following established categories in empirical software engineering.

Construct Validity. We measure effectiveness using fault detection rate and coverage metrics (line and branch). Fault detection rate relies on the accuracy of defect annotations in Defects4J [37] and GrowingBugs [33–35]; any missing or mislabeled defects may bias results. Coverage metrics depend on the instrumentation tool (Cobertura [58]); failures to instrument generated tests or exclusions in configuration could lead to under- or over-reporting.

Reference-Version Test Generation. All methods, including the baselines, generate tests (and, for ADVERTEST, mutants) on the *fixed* (reference) version of each program, and detection is scored by replaying the suite on the buggy version (Section 4.3). Generating every tool against a shared reference is the standard, fair basis for comparison, since a tool given the buggy version would simply encode the buggy behavior as expected. A consequence is that a mutant can occasionally land on a line introduced by the fix and entirely absent from the buggy version, as in the `try/except` block of the SWE-Rebench example in Section 5.4, where the hint is available only because generation occurs on the fixed version. We regard this as a genuine limitation rather than a confound: for most defects no mutant lands on fix-introduced code, and mutation instead helps through the coupling effect, whereby a suite that kills the seeded mutants is sensitive enough to expose the real fault. This reflects the premise of mutation testing, on which our framework is built: faults are detected as deviations from a program taken to be correct, with the real buggy version playing the same role that seeded mutants play during generation.

Internal Validity. We mitigate threats to internal validity through several measures. First, ADVERTEST is compared against four state-of-the-art baselines [15, 23, 49, 66], and statistical analysis is applied to ensure significance. Experiments are replicated across multiple LLMs to verify that observed gains are not artifacts of a specific model. Second, we address potential *data leakage*. Defects were intentionally selected from GrowingBugs entries added in December 2024, after the DeepSeek V3 [43] knowledge cutoff, to reduce the risk of models having prior exposure. Nevertheless, parts of the underlying projects may have existed earlier and could have been seen by LLMs, which may still introduce subtle leakage effects. Third, the RQ2 ablation (Section 5.2) uses a 50-bug random sub-sample of RQ1’s 200 bugs for compute reasons; we therefore read it only for component-wise comparisons under matched conditions, not as absolute effectiveness claims. Finally, while LLM nondeterminism presents an inherent validity threat, our diverse model selection and large-scale testing help ensure the robustness of reported results.

External Validity. Our evaluation spans both Java and Python, covering Defects4J [37] and GrowingBugs [33–35] for Java as well as 12 post-cutoff real-world issues from SWE-Rebench [5] for Python. These results provide initial evidence that ADVERTEST is not tied to a single language, benchmark, or test framework. Nevertheless, the current study still covers only two programming ecosystems and a limited set of tools (e.g., JUnit/Cobertura [58] and pytest/coverage.py); the Python generalization study is also smaller in scale than the Java evaluation. Regarding model selection,

we evaluate ADVERTEST on three representative LLMs to approximate broader applicability, but computational constraints prevent exhaustive evaluation of the entire model landscape. Caution is therefore still needed when extrapolating our findings to other languages, domains, testing frameworks, mutation engines, or future LLM architectures.

7 Future Work

Extending to Higher-Order Mutants. While our current framework focuses on first-order mutants for diagnosability and prompt reliability, a natural evolution is the generation of Higher-Order Mutants (HOMs) [32], either by combining LLM-generated first-order mutants or by prompting the LLM for multi-line edits directly. HOMs can simulate subtle faults that arise from the interaction of multiple defects, but the added complexity may not be cost-effective: the Coupling Effect hypothesis [47] suggests that test suites strong enough to kill simple mutants are already likely to kill complex ones. We leave a systematic study of this trade-off to future work.

RIP-Aware Survivor Feedback. Currently, when a mutant survives, we forward only its source-level diff to Agent $\mathcal{T}$ and let the LLM infer *why* the existing suite failed to kill it. A richer feedback channel would explicitly distinguish the three stages of the Reachability-Infection-Propagation (RIP) model: a survivor may go unreached (no test executes the mutated line), uninfected (executed but produces the same intermediate state), or unpropagated (state diverges but the divergence never reaches an observable assertion). Automating this RIP classification with lightweight dynamic analysis and presenting it alongside the mutant would let Agent $\mathcal{T}$ pick targeted remediations (extend coverage, strengthen assertions, or probe new outputs) rather than guessing. Dynamically computing the precise reason a mutant survives is a non-trivial program-analysis problem; we leave a thorough study of cost-effective RIP feedback to future work.

Beyond Unit-Test Generation. The core philosophy of ADVERTEST—two LLM agents iteratively probing each other’s blind spots under structural and mutation feedback—is, in principle, not tied to unit testing. A natural extension is compiler fuzzing (e.g., WhiteFox [68]), where the connection between a mutation in the compiler source and the input program that distinguishes it is far less direct than a method call against a focal class. Adapting the feedback loop to such domains would require richer signals than the source diff and coverage map we use today—likely including IR-level execution traces and differential output checks—but we see this as a promising avenue for generalizing the adversarial dual-agent paradigm to more complex testing tasks.

8 Conclusion

In this paper, we have presented ADVERTEST, an adversarial dual-agent framework to generate high-quality, robust unit tests. ADVERTEST combines a test generation agent ($\mathcal{T}$) with a mutant generation agent ($\mathcal{M}$), guiding their interaction through bidirectional feedback on mutation score and line coverage. The adversarial loop systematically exposes blind spots in the evolving test suite and drives both agents toward stronger fault detection capability. Experiments on two real-world benchmarks, Defects4J and GrowingBugs, demonstrate the practical benefits of this design. ADVERTEST improves the fault detection rate with statistical significance by 8.56% over the best existing LLM-based approach HITS and by 50.20% over the search-based tool EvoSuite, while maintaining a very competitive coverage rate against the state-of-the-art method HITS.

Data Availability

All code and data used in this study are publicly available at <https://github.com/jmueducn/AdverTest>.

Acknowledgment

This research is funded by the National Key Research and Development Program of China (Grant No. 2023YFB4503802) and the Natural Science Foundation of Shanghai (Grant No. 25ZR1401175).

References

- [1] Saranya Alagarsamy, Chakkrit Tantithamthavorn, and Aldeida Aleti. 2024. A3Test: Assertion-Augmented Automated Test case generation. *Information and Software Technology* 176 (2024), 107565. doi:10.1016/j.infsof.2024.107565
- [2] Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated Unit Test Improvement using Large Language Models at Meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (Porto de Galinhas, Brazil) (FSE 2024). Association for Computing Machinery, New York, NY, USA, 185–196. doi:10.1145/3663529.3663839
- [3] Juan Altmayer Pizzorno and Emery D. Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE128 (June 2025), 23 pages. doi:10.1145/3729398
- [4] Andrea Arcuri and Xin Yao. 2014. Co-evolutionary automatic programming for software development. *Information Sciences* 259 (2014), 412–432. doi:10.1016/j.ins.2009.12.019
- [5] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. SWE-Rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. In *Advances in Neural Information Processing Systems*, D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (Eds.), Vol. 38. Curran Associates, Inc. doi:10.52202/085713-0788
- [6] Morena Barboni, Filippo Lampa, Andrea Morichetta, Andrea Polini, and Edward Zulkoski. 2025. Mutant-Driven Test Generation for Ethereum Smart Contracts via LLMs. In *2025 IEEE International Conference on Artificial Intelligence Testing (AITest)*. 209–216. doi:10.1109/AITest66680.2025.00033
- [7] Moritz Beller, Georgios Gousios, Annibale Panichella, and Andy Zaidman. 2015. When, how, and why developers (do not) test in their IDEs. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering* (Bergamo, Italy) (ESEC/FSE 2015). Association for Computing Machinery, New York, NY, USA, 179–190. doi:10.1145/2786805.2786843
- [8] Moritz Beller, Georgios Gousios, and Andy Zaidman. 2015. How (Much) Do Developers Test?. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 2. 559–562. doi:10.1109/ICSE.2015.193
- [9] Chandrasekhar Boyapati, Sarfraz Khurshid, and Darko Marinov. 2002. Korat: automated testing based on Java predicates. *SIGSOFT Softw. Eng. Notes* 27, 4 (July 2002), 123–133. doi:10.1145/566171.566191
- [10] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation* (San Diego, California) (OSDI'08). USENIX Association, USA, 209–224.
- [11] Xia Cai and Michael R. Lyu. 2005. The effect of code coverage on fault detection under different testing profiles. In *Proceedings of the 1st International Workshop on Advances in Model-Based Testing* (St. Louis, Missouri) (A-MOST '05). Association for Computing Machinery, New York, NY, USA, 1–7. doi:10.1145/1083274.1083288
- [12] Pengyu Chang, Yixiong Fang, Silin Chen, Yuling Shi, Beijun Shen, and Xiaodong Gu. 2025. The replication package. <https://github.com/jmueducn/AdverTest>.
- [13] Mark Chen, Jerry Tworek, Heewoo Jun, et al. 2021. Evaluating Large Language Models Trained on Code. (2021). arXiv:2107.03374 [cs.LG] doi:10.48550/arXiv.2107.03374
- [14] Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, et al. 2025. Swe-exp: Experience-driven software issue resolution. *arXiv preprint arXiv:2507.23361* (2025). doi:10.48550/arXiv.2507.23361
- [15] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (Porto de Galinhas, Brazil) (FSE 2024). Association for Computing Machinery, New York, NY, USA, 572–576. doi:10.1145/3663529.3663801
- [16] Koen Claessen and John Hughes. 2000. QuickCheck: a lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming* (ICFP '00). Association for Computing Machinery, New York, NY, USA, 268–279. doi:10.1145/351240.351266
- [17] Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. 2016. PIT: a practical mutation testing tool for Java (demo). In *Proceedings of the 25th International Symposium on Software Testing and Analysis* (Saarbrücken, Germany) (ISSTA 2016). Association for Computing Machinery, New York, NY, USA, 449–452. doi:10.1145/2931037.2948707

- [18] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2024. Effective test generation using pre-trained Large Language Models and mutation testing. *Information and Software Technology* 171 (2024), 107468. doi:10.1016/j.infsof.2024.107468
- [19] DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, et al. 2025. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. arXiv:2512.02556 [cs.CL] doi:10.48550/arXiv.2512.02556
- [20] Renzo Degiovanni and Mike Papadakis. 2022. μ Bert: Mutation Testing using Pre-Trained Language Models. In *2022 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 160–169. doi:10.1109/ICSTW55395.2022.00039
- [21] Amirhossein Deljouyi, Roham Koohestani, Maliheh Izadi, and Andy Zaidman. 2025. Leveraging Large Language Models for Enhancing the Understandability of Generated Unit Tests. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 1449–1461. doi:10.1109/ICSE55347.2025.00032
- [22] R.A. DeMillo, R.J. Lipton, and F.G. Sayward. 1978. Hints on Test Data Selection: Help for the Practicing Programmer. *Computer* 11, 4 (1978), 34–41. doi:10.1109/C-M.1978.218136
- [23] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (Szeged, Hungary) (ESEC/FSE '11)*. Association for Computing Machinery, New York, NY, USA, 416–419. doi:10.1145/2025113.2025179
- [24] Gordon Fraser, Alessio Gambi, Marvin Kreis, and José Miguel Rojas. 2019. Gamifying a Software Testing Course with Code Defenders. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education (Minneapolis, MN, USA) (SIGCSE '19)*. Association for Computing Machinery, New York, NY, USA, 571–577. doi:10.1145/3287324.3287471
- [25] Aayush Garg, Renzo Degiovanni, Mike Papadakis, and Yves Le Traon. 2024. On the Coupling between Vulnerabilities and LLM-Generated Mutants: A Study on Vul4J Dataset. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. 305–316. doi:10.1109/ICST60714.2024.00035
- [26] Rahul Gopinath, Carlos Jensen, and Alex Groce. 2014. Code coverage for suite evaluation by developers. In *Proceedings of the 36th International Conference on Software Engineering (Hyderabad, India) (ICSE 2014)*. Association for Computing Machinery, New York, NY, USA, 72–82. doi:10.1145/2568225.2568278
- [27] Mark Harman, Jillian Ritchey, Inna Harper, Shubho Sengupta, Ke Mao, Abhishek Gulati, Christopher Foster, and Hervé Robert. 2025. Mutation-Guided LLM-based Test Generation at Meta. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (Clarion Hotel Trondheim, Trondheim, Norway) (FSE Companion '25)*. Association for Computing Machinery, New York, NY, USA, 180–191. doi:10.1145/3696630.3728544
- [28] Hadi Hemmati. 2015. How Effective Are Code Coverage Criteria?. In *2015 IEEE International Conference on Software Quality, Reliability and Security*. 151–156. doi:10.1109/QRS.2015.30
- [29] Chao Hu, Wenhao Zeng, Yuling Shi, Beijun Shen, and Xiaodong Gu. 2026. In Line with Context: Repository-Level Code Generation via Context Inlining. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE066 (June 2026), 23 pages. doi:10.1145/3797094
- [30] Yang Hu, Umair Z. Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. 2019. Re-Factoring Based Program Repair Applied to Programming Assignments. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 388–398. doi:10.1109/ASE.2019.00044
- [31] Kush Jain and Claire Le Goues. 2025. TestForge: Feedback-Driven, Agentic Test Suite Generation. *arXiv preprint arXiv:2503.14713* (2025). doi:10.48550/arXiv.2503.14713
- [32] Yue Jia and Mark Harman. 2009. Higher Order Mutation Testing. *Information and Software Technology* 51, 10 (2009), 1379–1393. Source Code Analysis and Manipulation, SCAM 2008. doi:10.1016/j.infsof.2009.04.016
- [33] Yanjie Jiang, Hui Liu, Xiaoqing Luo, Zhihao Zhu, Xiaye Chi, Nan Niu, Yuxia Zhang, Yamin Hu, Pan Bian, and Lu Zhang. 2022. BugBuilder: An Automated Approach to Building Bug Repository. *IEEE Transactions on Software Engineering* (2022), 1–22. doi:10.1109/TSE.2022.3177713
- [34] Yanjie Jiang, Hui Liu, Nan Niu, Lu Zhang, and Yamin Hu. 2021. Extracting Concise Bug-Fixing Patches from Human-Written Patches in Version Control Systems. In *IEEE/ACM 43rd International Conference on Software Engineering (ICSE 2021)*. IEEE Computer Society, Los Alamitos, CA, USA, 686–698. doi:10.1109/ICSE43902.2021.00069
- [35] Yanjie Jiang, Hui Liu, Yuxia Zhang, Weixing Ji, Hao Zhong, and Lu Zhang. 2022. Do Bugs Lead to Unnaturalness of Source Code?. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Singapore, Singapore) (ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 1085–1096. doi:10.1145/3540250.3549149
- [36] Paul C Jorgensen. 2013. *Software testing: a craftsman's approach*. Auerbach Publications. doi:10.1201/9781439889503
- [37] René Just, Darios Jalaali, and Michael D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (San Jose, CA, USA) (ISSTA 2014)*. Association for Computing Machinery, New York, NY, USA, 437–440. doi:10.1145/2610384.2628055

- [38] René Just, Franz Schweiggert, and Gregory M. Kapfhammer. 2011. MAJOR: An efficient and extensible tool for mutation analysis in a Java compiler. In *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*. 612–615. doi:10.1109/ASE.2011.6100138
- [39] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 919–931. doi:10.1109/ICSE48619.2023.00085
- [40] Han Li, Yuling Shi, Shaoxin Lin, Xiaodong Gu, Heng Lian, Xin Wang, Yantao Jia, Tao Huang, and Qianxiang Wang. 2025. Swe-debate: Competitive multi-agent debate for software issue resolution. *arXiv preprint arXiv:2507.23348* (2025). doi:10.48550/arXiv.2507.23348
- [41] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. 2017. QuixBugs: a multi-lingual program repair benchmark set based on the quixey challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity* (Vancouver, BC, Canada) (*SPLASH Companion 2017*). Association for Computing Machinery, New York, NY, USA, 55–56. doi:10.1145/3135932.3135941
- [42] Yalan Lin, Yingwei Ma, Rongyu Cao, Binhua Li, Fei Huang, Xiaodong Gu, and Yongbin Li. 2024. Llms as continuous learners: Improving the reproduction of defective code in software issues. *arXiv preprint arXiv:2411.13941* (2024). doi:10.48550/arXiv.2411.13941
- [43] Aixiu Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024). doi:10.48550/arXiv.2412.19437
- [44] Stephan Lukasczyk and Gordon Fraser. 2022. Pynguin: automated unit test generation for Python. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings* (Pittsburgh, Pennsylvania) (*ICSE '22*). Association for Computing Machinery, New York, NY, USA, 168–172. doi:10.1145/3510454.3516829
- [45] David R. MacIver, Zac Hatfield-Dodds, and Many Other Contributors. 2019. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software* 4, 43 (2019), 1891. doi:10.21105/joss.01891
- [46] Pengyu Nie, Rahul Banerjee, Junyi Jessy Li, Raymond J. Mooney, and Milos Gligoric. 2023. Learning Deep Semantics for Test Completion. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2111–2123. doi:10.1109/ICSE48619.2023.00178
- [47] A. Jefferson Offutt. 1992. Investigations of the software testing coupling effect. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (Jan. 1992), 5–20. doi:10.1145/125489.125473
- [48] OpenAI, Sandhini Agarwal, Lama Ahmad, et al. 2025. gpt-oss-120b and gpt-oss-20b Model Card. *arXiv:2508.10925* [cs.CL] doi:10.48550/arXiv.2508.10925
- [49] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: feedback-directed random testing for Java. In *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion* (Montreal, Quebec, Canada) (*OOPSLA '07*). Association for Computing Machinery, New York, NY, USA, 815–816. doi:10.1145/1297846.1297902
- [50] Nikitha Rao, Kush Jain, Uri Alon, Claire Le Goues, and Vincent J. Hellendoorn. 2023. CAT-LM Training Language Models on Aligned Code And Tests. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 409–420. doi:10.1109/ASE56229.2023.00193
- [51] G. Rothermel, R.H. Untch, Chengyun Chu, and M.J. Harrold. 1999. Test case prioritization: an empirical study. In *Proceedings IEEE International Conference on Software Maintenance - 1999 (ICSM'99)*. 'Software Maintenance for Business Change' (Cat. No.99CB36360). 179–188. doi:10.1109/ICSM.1999.792604
- [52] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105. doi:10.1109/TSE.2023.3334955
- [53] Yuling Shi, Yichun Qian, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. 2025. LongCodeZip: Compress Long Context for Code Language Models. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 141–153. doi:10.1109/ASE63991.2025.00020
- [54] Yuling Shi, Songsong Wang, Chengcheng Wan, Min Wang, and Xiaodong Gu. 2024. From code to correctness: Closing the last mile of code generation with hierarchical debugging. *arXiv preprint arXiv:2410.01215* (2024). doi:10.48550/arXiv.2410.01215
- [55] Yuling Shi, Hongyu Zhang, Chengcheng Wan, and Xiaodong Gu. 2025. Between Lines of Code: Unraveling the Distinct Patterns of Machine and Human Programmers. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 1628–1639. doi:10.1109/ICSE55347.2025.00005
- [56] Mohammed Latif Siddiq, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. 2024. Using Large Language Models to Generate JUnit Tests: An Empirical Study. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering* (Salerno, Italy)

- (EASE '24). Association for Computing Machinery, New York, NY, USA, 313–322. doi:10.1145/3661167.3661216
- [57] Philipp Straubinger, Marvin Kreis, Stephan Lukaszczuk, and Gordon Fraser. 2025. Mutation Testing via Iterative Large Language Model-Driven Scientific Debugging. In *2025 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 358–367. doi:10.1109/ICSTW64639.2025.10962485
- [58] The Cobertura Team. 2015. Cobertura. <https://github.com/cobertura/cobertura>.
- [59] Nikolai Tillmann and Jonathan de Halleux. 2008. Pex–White Box Test Generation for .NET. In *Tests and Proofs*, Bernhard Beckert and Reiner Hähnle (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 134–153. doi:10.1007/978-3-540-79124-9_10
- [60] Frank Tip, Jonathan Bell, and Max Schäfer. 2025. LLMorpheus: Mutation Testing Using Large Language Models. *IEEE Transactions on Software Engineering* 51, 6 (2025), 1645–1665. doi:10.1109/TSE.2025.3562025
- [61] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617* (2020). doi:10.48550/arXiv.2009.05617
- [62] Johannes Villmow, Jonas Depoix, and Adrian Ulges. 2021. ConTest: A Unit Test Completion Benchmark featuring Context. In *Proceedings of the 1st Workshop on Natural Language Processing for Programming (NLP4Prog 2021)*, Royi Lachmy, Ziyu Yao, Greg Durrett, Milos Gligoric, Junyi Jessy Li, Ray Mooney, Graham Neubig, Yu Su, Huan Sun, and Reut Tsarfaty (Eds.). Association for Computational Linguistics, Online, 17–25. doi:10.18653/v1/2021.nlp4prog-1.2
- [63] Bo Wang, Mingda Chen, Ming Deng, Youfang Lin, Mark Harman, Mike Papadakis, and Jie M. Zhang. 2026. A Comprehensive Study on Large Language Models for Mutation Testing. *ACM Trans. Softw. Eng. Methodol.* (March 2026). Just Accepted. doi:10.1145/3805038
- [64] Yuhang Wang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, Yuting Chen, Siyu Ye, Kai Cai, and Xiaodong Gu. 2026. SWE-Pruner: Self-Adaptive Context Pruning for Coding Agents. *arXiv preprint arXiv:2601.16746* (2026). doi:10.48550/arXiv.2601.16746
- [65] Yifei Wang, Ziteng Wang, Yuling Shi, Silin Chen, Xinrui Wang, Yueqi Wang, Beijun Shen, Linjing Li, Xiaodong Gu, Julian McAuley, and Daniel Dajun Zeng. 2026. Context Compression for LLM Agents: A Survey of Methods, Failure Modes, and Evaluation. *Preprints* (May 2026). doi:10.20944/preprints202605.2065.v1
- [66] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-coverage LLM-based Unit Test Generation via Method Slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 1258–1268. doi:10.1145/3691620.3695501
- [67] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 24824–24837. doi:10.52202/068431-1800
- [68] Chenyuan Yang, Yinlin Deng, Runyu Lu, Jiayi Yao, Jiawei Liu, Reyhaneh Jabbarvand, and Lingming Zhang. 2024. WhiteFox: White-Box Compiler Fuzzing Empowered by Large Language Models. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 296 (Oct. 2024), 27 pages. doi:10.1145/3689736
- [69] Wenhao Zeng, Xuteng Zhang, Yuling Shi, Chao Hu, Yuting Chen, Beijun Shen, and Xiaodong Gu. 2026. GlimpRouter: Efficient Collaborative Inference by Glimpsing One Token of Thoughts. In *Findings of the Association for Computational Linguistics: ACL 2026*, Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (Eds.). Association for Computational Linguistics, San Diego, California, United States, 17850–17864. doi:10.18653/v1/2026.findings-acl.885

Received 2026-01-30; accepted 2026-06-25